Spring Boot in the Cloud

Advanced Optimization Deep Dive

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com

Abstract

Spring Boot in the Cloud: Advanced Optimization Deep Dive

Spring Boot makes it easy to launch cloud-ready applications, but building truly lean and efficient cloud services requires going beyond the basics. In this three-hour deep dive, you'll immerse yourself in advanced Spring Boot techniques to optimize your applications for the cloud.

We will examine in depth how to reduce memory usage, improve startup times, and streamline your Spring Boot apps for modern cloud environments. Topics include Spring AOT, classpath exclusions, lazy beans, actuator configuration, custom JVM options, and additional state-of-the-art tools.

This session combines short presentations, live coding, and practical hands-on exercises. You will experiment with advanced configurations, observe real-time results, and troubleshoot performance challenges alongside the instructor. Real-world scenarios and best practices will help you immediately apply these techniques to your own projects.

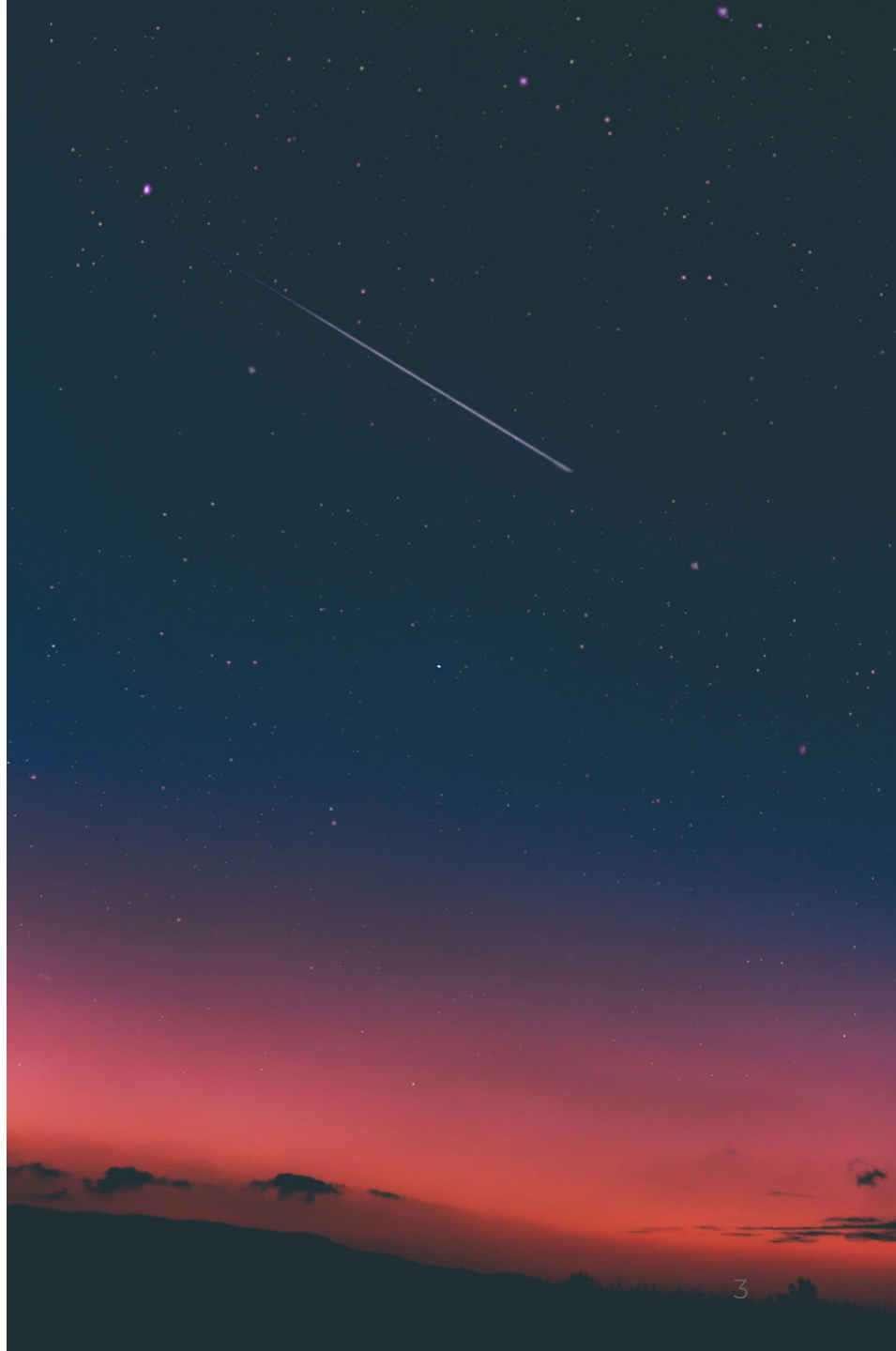
Whether you want to fine-tune your CI/CD pipeline or push Spring Boot to its limits in production, you will leave with actionable skills, a collection of code samples, and a deeper understanding of cloud-native application optimization. Get ready for an in-depth, practical exploration of building lean Spring Boot applications for the cloud.

Spring Boot in the Cloud

Advanced Optimization Deep Dive

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com



Spring Boot in the Cloud

Advanced Optimization Deep Dive

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com
bit.ly/48i5U9x

⚠ WARNING:

Numbers shown in this talk are not based on real data but only estimates and assumptions made by the author for educational purposes only.

Introduction

Patrick Baumgartner

Technical Agile Coach @ **42talents**

My focus is on the **development of software solutions *with* humans**.

Coaching, Architecture, Development, Reviews, and Training.

Lecturer @ **Zurich University of Applied Sciences ZHAW**

Co-Organizer of **Voxxed Days Zurich**, **JUG Switzerland**, Java Champion, Oracle ACE Pro Java ...

@patbaumgartner

What's the challenge?

Why does this matter?

I ❤️ Java 😉 & Spring Boot 🍃

Introduction to Optimization Goals

Exercises

Questions ...

Think about your current project.

- What would you optimise your application for?
- What are the key metrics?
- How would you measure them?

> 3' in pairs

Measures

- Use Case
- Build Time
- Image Size
- Startup Time
- CPU Usage
- Memory Usage
- Throughput / Requests per Second
- Latency / Response Time

Requirements

When Deploying to a Cloud

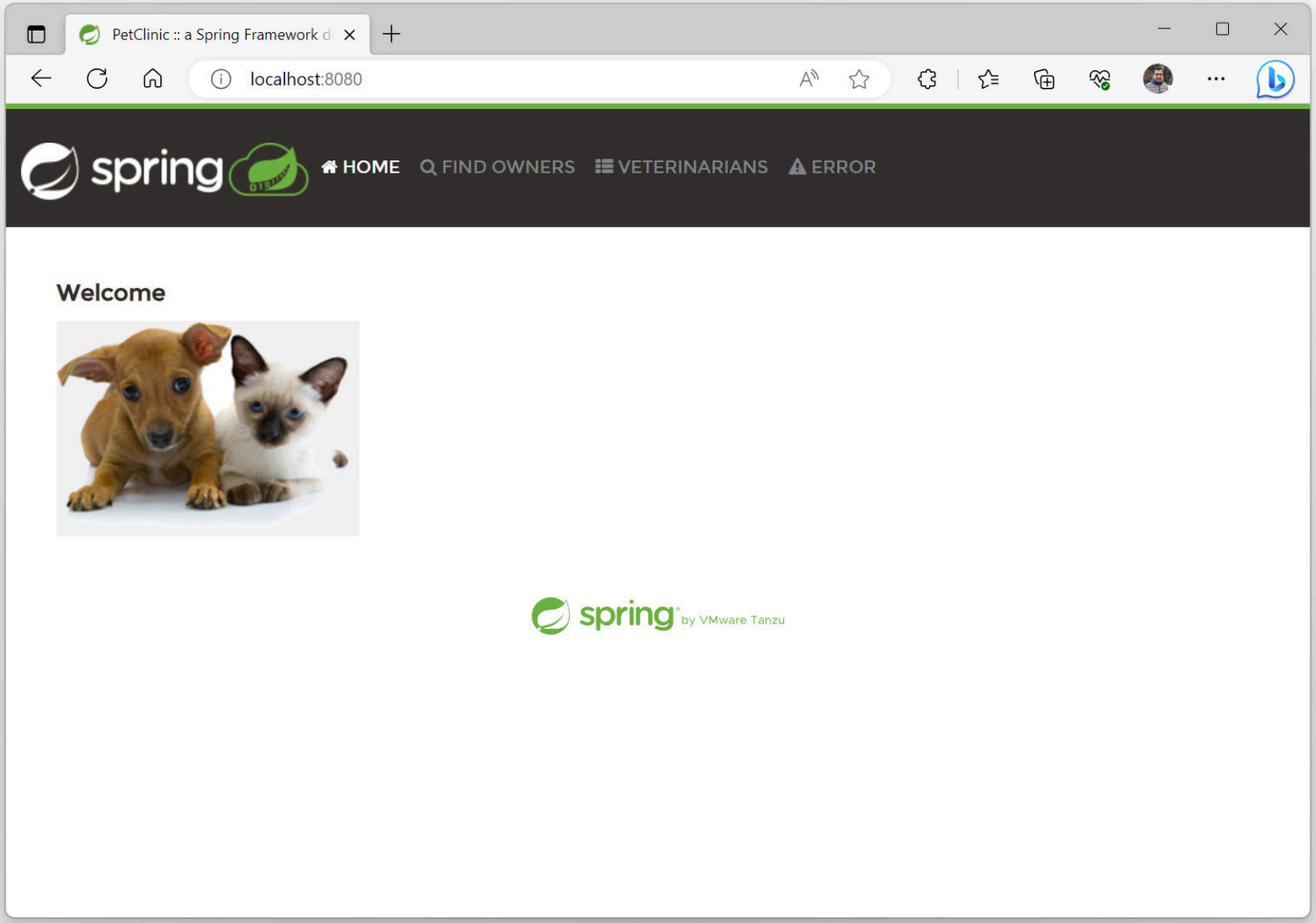
- How many vCPUs needs my application?
- How much RAM does it need?
- How much storage does it need?
- What technology stack should I use?
- What type of application do we build?

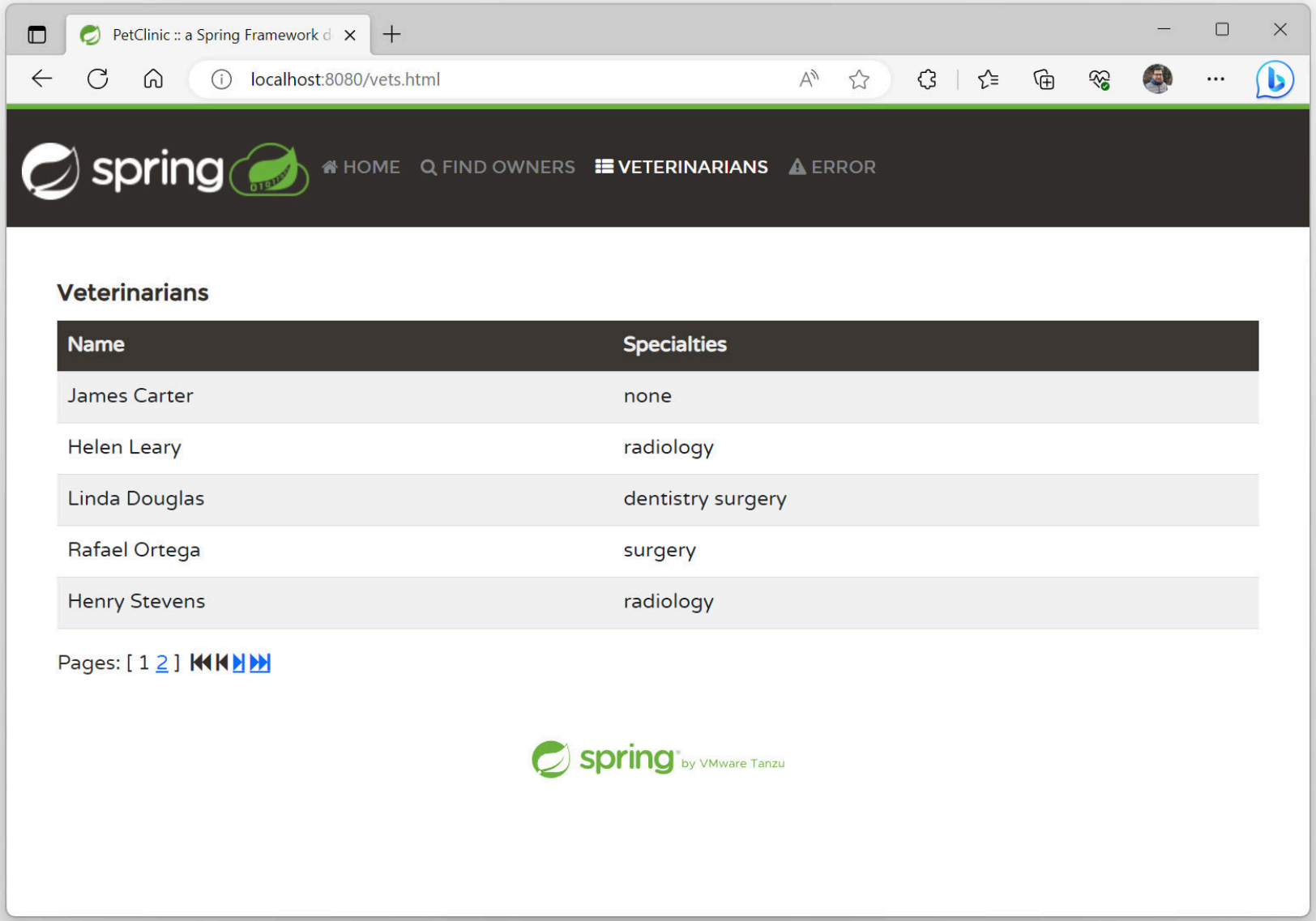
Agenda

Agenda

- Spring PetClinic & Baseline for comparison
- JVM Optimisations
- Spring Boot Optimisations
- Application Optimisations
- Other Runtimes
- Conclusions
- Some simple optimisations applied (OpenJDK examples)

Spring PetClinic


PetClinic :: a Spring Framework d
localhost:8080
spring
HOME
FIND OWNERS
VETERINARIANS
ERROR
Welcome
spring by VMware Tanzu


PetClinic :: a Spring Framework d
localhost:8080/vets.html
spring
HOME
FIND OWNERS
VETERINARIANS
ERROR
Veterinarians
Name
Specialties
James Carter
none
Helen Leary
radiology
Linda Douglas
dentistry surgery
Rafael Ortega
surgery
Henry Stevens
radiology
Pages: [1 2]
spring by VMware Tanzu

Spring Petclinic Community

- spring-framework-petclinic
- spring-petclinic-angular(js)
- spring-petclinic-rest
- spring-petclinic-graphql
- spring-petclinic-microservices
- spring-petclinic-data-jdbc
- spring-petclinic-cloud
- spring-petclinic-mustache
- spring-petclinic-kotlin
- spring-petclinic-reactive
- spring-petclinic-hilla
- spring-petclinic-angularjs
- spring-petclinic-vaadin-flow
- spring-petclinic-reactjs
- spring-petclinic-htmx
- spring-petclinic-istio
- ...

Projects on GitHub: https://github.com/spring-petclinic

NO!

The official Spring PetClinic! 🐾🏥

Based on Spring Boot, Caffeine, Thymeleaf, Spring Data JPA, H2 and Spring MVC ...

Project on GitHub: https://github.com/spring-projects/spring-petclinic

Limitations

Limitations

- Docker stats prints wrong values for memory usage, use docker top instead. (See: https://quarkus.io/guides/performance-measure#measuring-memory-correctly-on-docker)

```
docker top <CONTAINER ID> -o pid,rss,args
```

- Thermal throttling of your notebook CPU may affect your results

Benchmark Steps

Using oha

Build with Maven

Set the Java version and build the OCI container image.

```
sdk use java 17.0.18-librca
java -version

build_time=$(./mvnw clean spring-boot:build-image \
  | grep "Total time:" \
  | awk '{print $4 $5}')

echo -e  "\033[0;32mBuild time: $build_time\033[0m"

image_size=$(docker images \
  | grep "spring-petclinic" \
  | awk '{print $7}')

echo -e  "\033[0;32mImage Size: $image_size\033[0m"
```

Container Startup Time

Run the container with limited memory and measure the startup time.

```
docker container run -d --rm --memory="1g" -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT

# Get the container ID of the running container
container_id=$(docker ps -q)

startup_time=$(docker logs "$container_id"
  | grep "Started" \
  | awk '{print $14 $15}')

echo -e  "\033[0;32mStartup time: $startup_time\033[0m"
```

Memory Usage after Startup

Extract the memory usage of the running container. Convert `rss` to MB.

```
memory=$(docker top $(docker ps -lq) -o pid,rss,args \
  | tail -n +2 \
  | awk '{$2=int($2/1024)"MB";}{ print $2;}')

echo -e  "\033[0;32mStartup memory usage: $memory\033[0m"
```

Warm-up Application

Warm up the application with for 60s, json output and no UI.

```
oha --no-tui -z60s --disable-keepalive -j "http://localhost:8080/owners?page=2"
```

Load-Test Application

Load test during 60s, json output and no UI.

```
throughput=$(oha --no-tui -z60s --disable-keepalive -j "http://localhost:8080/owners?page=2" 2>&1 \
  | tee $filename \
  | jq '.summary.requestsPerSec')

echo -e  "\033[0;32mAvg Throughput: $throughput Requests/sec\033[0m"
```

Memory Usage after Load-Test

Extract the memory usage of the running container. Convert `rss` to MB.

```
memory=$(docker top $(docker ps -lq) -o pid,rss,args \
  | tail -n +2 \
  | awk '{$2=int($2/1024)"MB";}{ print $2;}')

echo -e  "\033[0;32mEnd memory usage: $memory\033[0m"
```

Optimisation Experiments

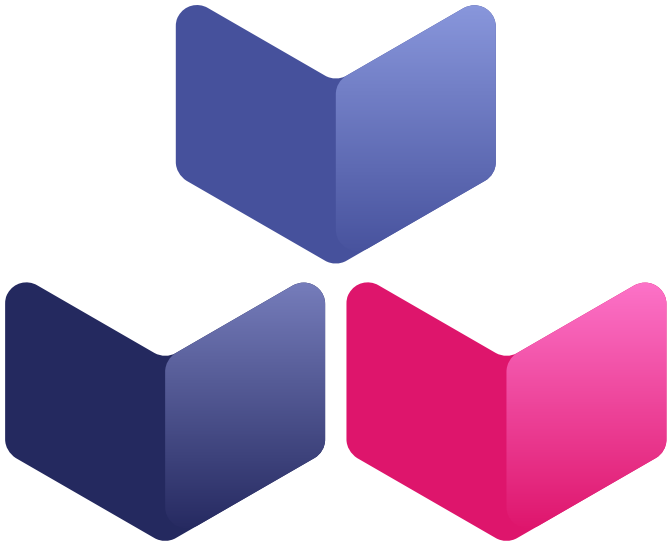
Baseline

Technology Stack

- OCI Container built with Buildpacks
- Java JDK 17 LTS
- Spring Boot 4.0
- Testcontainers
- DB migration using SQL scripts

Examination

- Build time
- Startup time
- Resource usage
- Container image size
- Throughput



Buildpacks.io

paketo

buildpacks

Your app,
in your favorite language,
ready to run in the cloud

```
Calculating JVM memory based on 22958184K available memory
For more information on this calculation, see https://paketo.io/docs/reference/java-reference/#memory-calculator
Calculated JVM Memory Configuration: -XX:MaxDirectMemorySize=10M -Xmx22316444K -XX:MaxMetaspaceSize=129739K -XX:ReservedCodeCacheSize=240M -Xss1M
(Total Memory: 22958184K, Thread Count: 250, Loaded Class Count: 20492, Headroom: 0%)
Enabling Java Native Memory Tracking
Adding 146 container CA certificates to JVM truststore
Spring Cloud Bindings Enabled
Picked up JAVA_TOOL_OPTIONS: -Djava.security.properties=/layers/paketo-buildpacks_bellsoft-liberica/java-security-properties/java-security.properties
  -XX:+ExitOnOutOfMemoryError -XX:MaxDirectMemorySize=10M -Xmx22316444K -XX:MaxMetaspaceSize=129739K -XX:ReservedCodeCacheSize=240M -Xss1M
  -XX:+UnlockDiagnosticVMOptions -XX:NativeMemoryTracking=summary -XX:+PrintNMTStatistics -Dorg.springframework.cloud.bindings.boot.enable=true


             |\      _,,,--,,_
            /,`.-'`'   ._  \-;;,_
 _______ __|,4-  ) )_   .;.(__`'-'__     ___ __    _ ___ _______
|       | '---''(_/._)-'(_\_)   |   |   |   |  |  | |   |       |
|    _  |    ___|_     _|       |   |   |   |   |_| |   |       | __ _ _
|   |_| |   |___  |   | |       |   |   |   |       |   |       | \ \ \ \
|    ___|    ___| |   | |      _|   |___|   |  _    |   |      _|  \ \ \ \
|   |   |   |___  |   | |     |_|       |   | | |   |   |     |_    ) ) ) )
|___|   |_______| |___| |_______|_______|___|_|  |__|___|_______|  / / / /
 ==========================================================/_/_/_/

:: Built with Spring Boot :: 4.0.5


2026-03-29T18:00:07.524Z  INFO 1 --- [           main] o.s.s.petclinic.PetClinicApplication     : Starting PetClinicApplication v4.0.0-SNAPSHOT using Java 17.0.18
  with PID 1 (/workspace/BOOT-INF/classes started by cnb in /workspace)
2026-03-29T18:00:07.528Z  INFO 1 --- [           main] o.s.s.petclinic.PetClinicApplication     : No active profile set, falling back to 1 default profile: "default"
2026-03-29T18:00:08.469Z  INFO 1 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-03-29T18:00:08.527Z  INFO 1 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 47 ms. Found 3 JPA repository interfaces.
2026-03-29T18:00:09.365Z  INFO 1 --- [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat initialized with port 8080 (http)
2026-03-29T18:00:09.380Z  INFO 1 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-03-29T18:00:09.381Z  INFO 1 --- [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/11.0.18]
2026-03-29T18:00:09.440Z  INFO 1 --- [           main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1860 ms
2026-03-29T18:00:09.733Z  INFO 1 --- [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2026-03-29T18:00:09.937Z  INFO 1 --- [           main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:8a2f72dc-247f-4908-98aa-3a8d6d1fd45a user=SA
2026-03-29T18:00:09.940Z  INFO 1 --- [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2026-03-29T18:00:10.124Z  INFO 1 --- [           main] org.hibernate.orm.jpa                    : HHH008540: Processing PersistenceUnitInfo [name: default]
2026-03-29T18:00:10.182Z  INFO 1 --- [           main] org.hibernate.orm.core                   : HHH000001: Hibernate ORM core version 7.2.4.Final
2026-03-29T18:00:08.145Z  INFO 1 --- [           main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-03-29T18:00:08.205Z  INFO 1 --- [           main] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
        Database JDBC URL [jdbc:h2:mem:8a2f72dc-247f-4908-98aa-3a8d6d1fd45a]
        Database driver: H2 JDBC Driver
        Database dialect: H2Dialect
        Database version: 2.4.240
        Default catalog/schema: 8A2F72DC-247F-4908-98AA-3A8D6D1FD45A/PUBLIC
        Autocommit mode: undefined/unknown
        Isolation level: READ_COMMITTED [default READ_COMMITTED]
        JDBC fetch size: 100
        Pool: DataSourceConnectionProvider
        Minimum pool size: undefined/unknown
        Maximum pool size: undefined/unknown
2026-03-29T18:00:09.147Z  INFO 1 --- [           main] org.hibernate.orm.core                   : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-03-29T18:00:09.153Z  INFO 1 --- [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-03-29T18:00:09.250Z  INFO 1 --- [           main] o.s.d.j.r.query.QueryEnhancerFactories   : Hibernate is in classpath; If applicable, HQL parser will be used.
2026-03-29T18:00:10.577Z  INFO 1 --- [           main] o.s.b.a.e.web.EndpointLinksResolver      : Exposing 13 endpoints beneath base path '/actuator'
2026-03-29T18:00:10.688Z  INFO 1 --- [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with context path '/'
2026-03-29T18:00:10.701Z  INFO 1 --- [           main] o.s.s.petclinic.PetClinicApplication     : Started PetClinicApplication in 3.621 seconds (process running for 6.498)
```

1000x better than your regular Dockerfile  ...

... more secure  and maintained by the Buildpacks community.

See also: https://buildpacks.io/ and https://www.cncf.io/projects/buildpacks/

Friends don't let friends write Dockerfiles!

@starbuxman

Benchmarks

Benchmarks

- Build
  - Maven build time
  - Artifact / Container Image size
- Startup
  - Startup time
  - Memory usage
- Throughput & Latency
  - `oha --no-tui -z60s --disable-keepalive <URL>`
  - 60s warmup, 60s measurement
  - Docker container with 2 vCPU and 1 GB RAM

No Optimizing - Baseline

No Optimizing - Baseline JDK 17

- Spring PetClinic (no adjustments)
- Bellsoft Liberica JDK 17.0.18
- Java Memory Calculator

```
sdk use java 17.0.18-librca

mvn spring-boot:build-image

docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms

No Optimizing - Baseline JDK 21

- Spring PetClinic (JDK 21 adjustments)
- Bellsoft Liberica JDK 21.0.10
- Java Memory Calculator

```
sdk use java 21.0.10-librca

mvn -Djava.version=21 spring-boot:build-image \
    -Dspring-boot.build-image.imageName=spring-petclinic:4.0.0-SNAPSHOT-jdk21

docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT-jdk21
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21	64s	307MB	4.943s	273MB	257/s	383MB	184ms	292ms	404ms	702ms	994ms

No Optimizing - Baseline JDK 25

- Spring PetClinic (JDK 25 adjustments)
- Bellsoft Liberica JDK 25.0.2
- Java Memory Calculator

```
sdk use java 25.0.2-librca

mvn -Djava.version=25 spring-boot:build-image \
    -Dspring-boot.build-image.imageName=spring-petclinic:4.0.0-SNAPSHOT-jdk25

docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT-jdk25
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 25	66s	376MB	4.887s	270MB	227/s	438MB	194ms	305ms	464ms	771ms	1040ms

No Optimizing - Baseline JDK 26

- Spring PetClinic (JDK 26 adjustments)
- Bellsoft Liberica JDK 26
- Java Memory Calculator

```
sdk use java 26-librca

mvn -Djava.version=26 spring-boot:build-image \
    -Dspring-boot.build-image.imageName=spring-petclinic:4.0.0-SNAPSHOT-jdk26

docker run -p 8080:8080 -t spring-petclinic:4.0.0-SNAPSHOT-jdk26
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 26	66s	382MB	4.939s	269MB	256/s	395MB	184ms	293ms	404ms	694ms	908ms

JVM Optimisations

-noverify

-noverify

The verifier is turned off because some of the bytecode rewriting stretches the meaning of some bytecodes - in a way that doesn't bother the JVM, but does bother the verifier.

Warning: The `-Xverify:none` and `-noverify` options are deprecated in JDK 13 and are likely to be removed in a future release.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-noverify" \
  -t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	-	-	4.422s	243MB	220/s	355MB	198ms	307ms	468ms	791ms	1103ms
Java 21	-	-	4.193s	252MB	235/s	372MB	192ms	300ms	440ms	782ms	1010ms
Java 25	-	-	4.282s	252MB	234/s	417MB	192ms	301ms	435ms	727ms	971ms

-XX:+UseCompactObjectHeaders

-XX:+UseCompactObjectHeaders

The Compact Object Headers (JEP 384) feature reduces the memory footprint of Java objects by using a more compact representation for object headers. This can lead to improved cache performance and reduced memory usage.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:+UseCompactObjectHeaders" \
  -t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
⏱️ Java 25	66s	376MB	4.887s	270MB	227/s	438MB	194ms	305ms	464ms	771ms	1040ms
Java 25	-	-	5.028s	264MB	194/s	441MB	209ms	366ms	504ms	828ms	1082ms

-XX:TieredStopAtLevel=1

-XX:TieredStopAtLevel=1

Tiered compilation is enabled by default (since Java 8). Unless explicitly specified, the JVM decides which JIT compiler to use based on our CPU. For multi-core processors or 64-bit VMs, the JVM will select C2.
To disable C2 and use only the C1 compiler with no profiling overhead, we can use the `-XX:TieredStopAtLevel=1` parameter.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:TieredStopAtLevel=1" \
  -t spring-petclinic:4.0.0-SNAPSHOT
```

It will slow down the JIT later at the expense of the saved startup time!

	Build	Image	Startup	Initial RAM	T...	RAM	50%	75%	90%	99%	99.9%
⏱ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	-	-	3.895s	222MB	152/s	296MB	308ms	406ms	503ms	699ms	814ms
Java 21	-	-	4.058s	232MB	146/s	303MB	316ms	417ms	517ms	710ms	890ms
Java 25	-	-	3.75s	227MB	174/s	385MB	288ms	394ms	499ms	703ms	892ms

-XX:+UseParallelGC

-XX:+UseParallelGC

The Parallel Garbage Collector (Parallel GC) is a high-throughput garbage collector that is designed to take advantage of multiple processors. It performs minor garbage collections in parallel, which can lead to shorter pause times and improved application performance.

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:+UseParallelGC" \
  -t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	T…	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21	-	-	4.886s	349MB	249/s	474MB	190ms	294ms	407ms	704ms	995ms
Java 25	-	-	4.728s	348MB	242/s	472MB	190ms	299ms	432ms	720ms	931ms
Java 26	-	-	4.709s	305MB	253/s	452MB	187ms	294ms	403ms	691ms	897ms

-XX:+UseZGC -XX:+ZGenerational

-XX:+UseZGC -XX:+ZGenerational

The Z Garbage Collector (ZGC) is a scalable, low-latency garbage collector. ZGC performs all expensive work concurrently without stopping application threads for more than 10ms, making it suitable for applications that require low latency and/or use a very large heap (multi-terabyte).

```
docker run -p 8080:8080 -e "JAVA_TOOL_OPTIONS=-XX:+UseZGC -XX:+ZGenerational" \
  -t spring-petclinic:4.0.0-SNAPSHOT
```

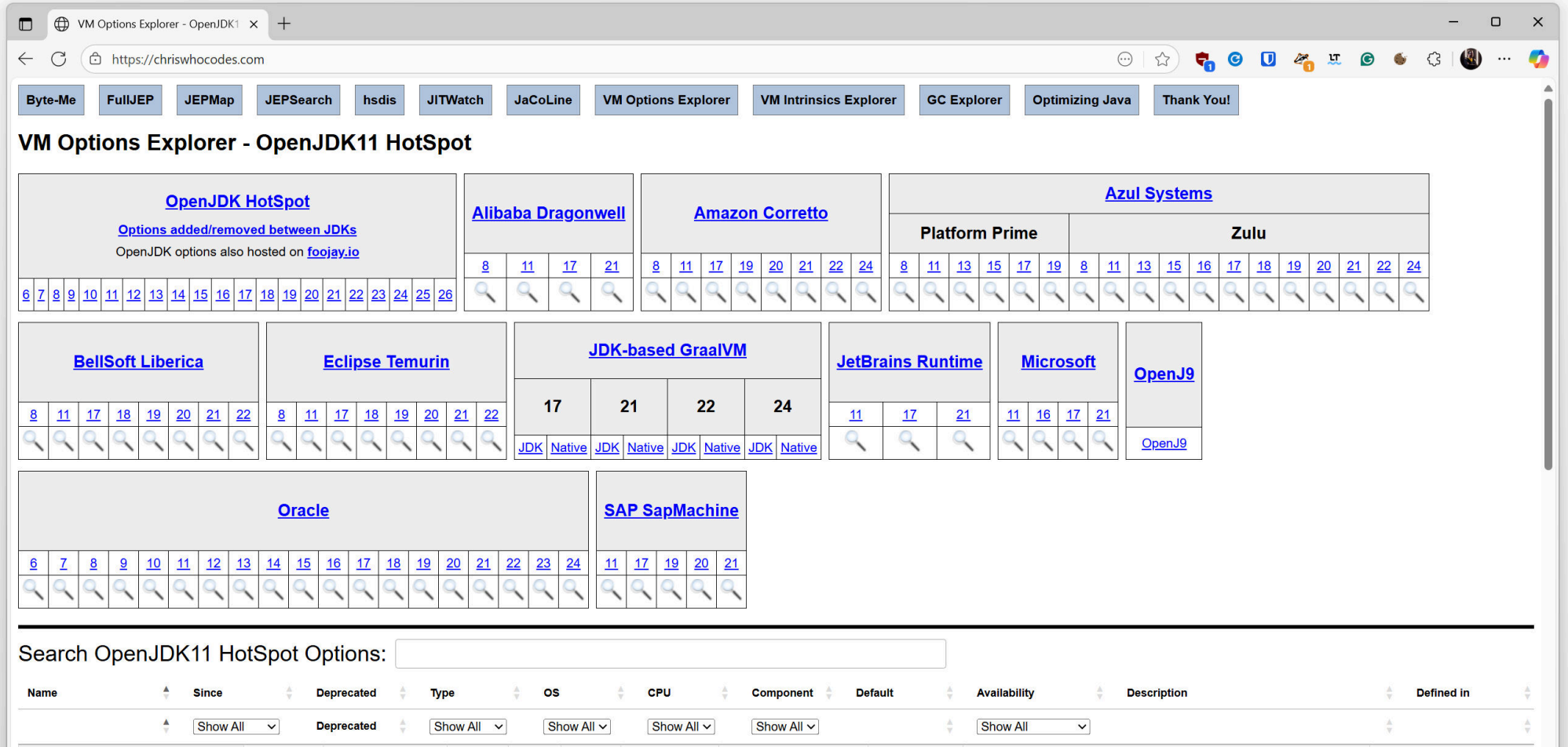
	Build	Image	Startup	Initial RAM	T…	RAM	50%	75%	90%	99%	99.9%
Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21	-	-	5.17s	524MB	225/s	673MB	195ms	306ms	490ms	803ms	1157ms
Java 25	-	-	5.237s	495MB	213/s	655MB	199ms	332ms	495ms	826ms	1099ms

See also: https://wiki.openjdk.org/display/zgc/Main

VM Options Explorer

VM Options Explorer

https://chriswhocodes.com



Spring Boot Optimisations

Lazy Spring Beans

Lazy Spring Beans (1)

Configure lazy initialisation throughout your application. A Spring Boot property makes all beans lazy by default, initialising them only when needed. You can use `@Lazy` to override this behaviour, e.g. `@Lazy(false)`.

```
docker run -p 8080:8080  \
  -e spring.main.lazy-initialization=true  \
  -e spring.data.jpa.repositories.bootstrap-mode=lazy \
  -t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	-	-	4.501s	239MB	234/s	356MB	192ms	301ms	438ms	740ms	1002ms

Lazy Spring Beans (2)

Pros

- Faster startup useful in cloud environments
- Application startup is a CPU-intensive task. Spreads load over time

Cons

- Initial requests may take longer
- Class loading problems and misconfigurations not detected at startup
- Beans creation errors only be found when the bean is loaded

Fixing Spring Boot Config Location

Fixing Spring Boot Config Location

Determine the location of the Spring Boot configuration file(s). Considered in the following order (`application.properties` and `YAML` variants)

- Application properties packaged in your jar
- Profile-specific application properties packaged within your jar
- Application properties outside your packaged jar
- Profile-specific application properties outside your packaged jar

```
docker run -p 8080:8080 -e spring.config.location=classpath:application.properties \
    t spring-petclinic:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	-	-	5.099s	257MB	232/s	358MB	192ms	302ms	456ms	758ms	1013ms

No Spring Boot Actuators

No Spring Boot Actuators

Don't use actuators if you can afford not to 😊.

- Number of Spring Beans
  - Spring Pet Clinic with actuators: 457
  - Spring Pet Clinic without actuators: 282 🔥

```
sdk use java 17.0.18-librca

mvn spring-boot:build-image
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	64s	277MB	4.466s	246MB	249/s	361MB	185ms	295ms	406ms	701ms	972ms
Java 21	63s	305MB	4.316s	252MB	265/s	360MB	176ms	289ms	400ms	694ms	985ms
Java 25	64s	373MB	4.442s	250MB	256/s	455MB	182ms	292ms	405ms	693ms	926ms

Ahead-of-Time Processing (AOT)

Ahead-of-Time Processing (AOT) (1)

Spring AOT is a process that analyses your application at build time and generates an optimised version of it.

As the `BeanFactory` is fully prepared at build time, conditions are also evaluated. E.g. in Configurations, Component- & Entity-Scan, `@Profile`, `@Conditional`, etc.

Spring AOT serves as a replacement for `spring-context-indexer` since Spring Framework 6.1 and Spring Boot 3.2.

Ahead-of-Time Processing (AOT) (2)

```
<plugin>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
      <image>
        <env>
          <BP_SPRING_AOT_ENABLED>true</BP_SPRING_AOT_ENABLED>
        </env>
      </image>
    </configuration>
    <executions>
      <execution>
          <id>process-aot</id>
          <goals>
              <goal>process-aot</goal>
          </goals>
      </execution>
    </executions>
</plugin>
```

Ahead-of-Time Processing (AOT) (3)

We create a new container image with the AOT-processed application. The Buildpack enables the property `spring.aot.enabled=true`

```
sdk use java 17.0.18-librca

mvn spring-boot:build-image

docker run -p 8080:8080 -e spring.aot.enabled=true \
  -t spring-petclinic-aot:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	71s	281MB	4.095s	242MB	227/s	352MB	195ms	303ms	479ms	786ms	1031ms
Java 21	71s	309MB	3.912s	245MB	251/s	368MB	187ms	294ms	407ms	706ms	984ms
Java 25	70s	377MB	4.03s	237MB	245/s	541MB	189ms	296ms	409ms	700ms	909ms

Disabling JMX

Disabling JMX

JMX is `spring.jmx.enabled=false` by default in Spring Boot since 2.2.0 and later. Setting `BPL_JMX_ENABLED=true` and `BPL_JMX_PORT=9999` on the container will add the following arguments to the `java` command.

```
-Djava.rmi.server.hostname=127.0.0.1
-Dcom.sun.management.jmxremote.authenticate=false
-Dcom.sun.management.jmxremote.ssl=false
-Dcom.sun.management.jmxremote.port=9999
-Dcom.sun.management.jmxremote.rmi.port=9999
```

```
docker run -p 8080:8080 -p 9999:9999 \
  -e BPL_JMX_ENABLED=false \
  -e BPL_JMX_PORT=9999 \
  -e spring.jmx.enabled=false \
  -t spring-petclinic:4.0.0-SNAPSHOT
```

I ❤️ Spring Boot 🍃 & Buildpacks 🚀

Application Optimisations

Dependency Cleanup

Dependency Cleanup (2)

DepClean detects all unused dependencies declared in the `pom.xml` file of a project and creates a `pom-debloated.xml`. The generated report shows possible unused dependencies.

```
<plugin>
  <groupId>se.kth.castor</groupId>
  <artifactId>depclean-maven-plugin</artifactId>
  <version>2.2.0-SNAPSHOT</version>
  <executions>
    <execution>
      <goals>
        <goal>depclean</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

```
mvn se.kth.castor:depclean-maven-plugin:2.2.0-SNAPSHOT:depclean -DfailIfUnusedDirect=true -DignoreScopes=provided,test,runtime,system,import
```

```
-------------------------------------------------------
 D E P C L E A N   A N A L Y S I S   R E S U L T S
-------------------------------------------------------
USED DIRECT DEPENDENCIES [10]:
        com.h2database:h2:2.4.240:runtime (2 MB)
        com.mysql:mysql-connector-j:9.6.0:runtime (2 MB)
        org.postgresql:postgresql:42.7.10:runtime (1 MB)
        com.github.ben-manes.caffeine:caffeine:3.2.3:runtime (996 KB)
        jakarta.xml.bind:jakarta.xml.bind-api:4.0.4:compile (128 KB)
        org.springframework.boot:spring-boot-testcontainers:4.0.5:test (74 KB)
        ...
USED TRANSITIVE DEPENDENCIES [107]:
        org.testcontainers:testcontainers:2.0.3:test (16 MB)
        org.hibernate.orm:hibernate-core:7.2.4.Final:compile (13 MB)
        net.bytebuddy:byte-buddy:1.17.8:runtime (8 MB)
        org.apache.tomcat.embed:tomcat-embed-core:11.0.18:compile (3 MB)
        com.github.docker-java:docker-java-transport-zerodep:3.7.0:test (2 MB)
        org.springframework:spring-web:7.0.5:compile (2 MB)
        ...
USED INHERITED DIRECT DEPENDENCIES [0]:
USED INHERITED TRANSITIVE DEPENDENCIES [0]:
POTENTIALLY UNUSED DIRECT DEPENDENCIES [14]:
        org.webjars.npm:bootstrap:5.3.8:runtime (1 MB)
        org.webjars.npm:font-awesome:4.7.0:runtime (665 KB)
        org.springframework.boot:spring-boot-devtools:4.0.5:compile (208 KB)
        org.springframework.boot:spring-boot-docker-compose:4.0.5:test (123 KB)
        org.springframework.boot:spring-boot-starter-actuator:4.0.5:compile (4 KB)
        ...
POTENTIALLY UNUSED TRANSITIVE DEPENDENCIES [34]:
        commons-codec:commons-codec:1.19.0:test (365 KB)
        org.springframework.boot:spring-boot-actuator:4.0.5:compile (348 KB)
        org.yaml:snakeyaml:2.5:compile (332 KB)
        org.attoparser:attoparser:2.0.7.RELEASE:compile (240 KB)
        org.springframework.boot:spring-boot-micrometer-metrics:4.0.5:compile (235 KB)
        org.thymeleaf:thymeleaf-spring6:3.1.3.RELEASE:compile (184 KB)
        org.hdrhistogram:HdrHistogram:2.2.2:runtime (173 KB)
        org.unbescape:unbescape:1.1.6.RELEASE:compile (169 KB)
        org.springframework.boot:spring-boot-web-server:4.0.5:compile (165 KB)
        org.springframework.boot:spring-boot-health:4.0.5:compile (150 KB)
        ...
POTENTIALLY UNUSED INHERITED DIRECT DEPENDENCIES [0]:
POTENTIALLY UNUSED INHERITED TRANSITIVE DEPENDENCIES [0]:
[INFO] Analysis done in 0min 27s
```

Dependency Cleanup (2)

But there are some challenges:

- Component & Entity Scanning through Classpath Scanning
- Spring Boot uses `META-INF/spring-boot/org.springframework.boot.autoconfigure.AutoConfiguration.imports`
- Spring XML Configuration and `web.xml`

```
sdk use java 17.0.18-librca

mvn spring-boot:build-image
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	37s	270MB	4.278s	245MB	256/s	349MB	177ms	292ms	402ms	700ms	969ms
Java 21	36s	297MB	4.024s	245MB	248/s	372MB	179ms	296ms	409ms	735ms	1002ms
Java 25	37s	366MB	4.086s	240MB	277/s	437MB	142ms	280ms	393ms	667ms	901ms

More Application Optimisations

More Application Optimisations

- Application Optimisations
- Better Connection Pooling
- Improved Data Caching
- Database Queries Optimisations
- Batch Processing Optimisations
- Reduce Garbage Collection
- Monitor and Tune JVM Settings
- Reduce Overall Memory Footprint
- Async Logging

Even More JVM Optimisations

JLink

JLink (1)

`jlink` assembles and optimises a set of modules and their dependencies into a custom runtime image for your application.

```
$ jlink \
  --add-modules java.base, ... \
  --strip-debug \
  --no-man-pages \
  --no-header-files \
  --compress=2 \
  --output /javaruntime
```

```
$ /javaruntime/bin/java HelloWorld
Hello, World!
```

JLink (2)

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
      <image>
          <env>
              <BP_JVM_JLINK_ENABLED>true</BP_JVM_JLINK_ENABLED>
          </env>
      </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	74s	191MB	5.003s	264MB	238/s	374MB	190ms	299ms	433ms	735ms	1002ms
Java 21	74s	199MB	4.943s	284MB	258/s	388MB	184ms	291ms	404ms	700ms	986ms
Java 25	72s	201MB	4.928s	282MB	268/s	386MB	178ms	286ms	395ms	652ms	897ms

Java Dependency Analysis Tool

`jdeps` is a Java command-line tool that analyzes class and jar files to list the package-level or class-level dependencies, helping developers understand module dependencies and improve code modularity.

```
jar xvf target/spring-petclinic-4.0.0-SNAPSHOT.jar

jdeps --ignore-missing-deps -q  \
    --recursive  \
    --multi-release 17  \
    --print-module-deps  \
    --class-path 'BOOT-INF/lib/*'  \
    target/spring-petclinic-4.0.0-SNAPSHOT.jar
```

```
java.base,java.compiler,java.desktop,java.instrument,java.net.http,java.prefs,java.rmi,java.scripting,java.security.jgss,java.sql.rowset,jdk.jfr,jdk.management,jdk.unsupported
```

JLink (3)

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
    <configuration>
        <image>
            <env>
                <BP_JVM_JLINK_ENABLED>true</BP_JVM_JLINK_ENABLED>
                <BP_JVM_JLINK_ARGS>--add-modules java.base,java.compiler,java.desktop,java.instrument,
                java.net.http,java.prefs,java.rmi,java.scripting,java.security.jgss,java.sql.rowset,
                jdk.jfr,jdk.management,jdk.unsupported --compress=2 --no-header-files --no-man-pages
                --strip-debug</BP_JVM_JLINK_ARGS>
            </env>
        </image>
    </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	72s	179MB	5.13s	266MB	232/s	368MB	194ms	302ms	443ms	763ms	1041ms
Java 21	71s	187MB	4.964s	283MB	256/s	390MB	185ms	291ms	403ms	701ms	990ms
Java 25	70s	189MB	4.949s	277MB	262/s	378MB	181ms	289ms	399ms	685ms	903ms

Application Class Data Sharing (App CDS)

App CDS (1)

Class Data Sharing (CDS) is a JVM feature that can help reduce the startup time and memory footprint of Java applications. It allows classes to be pre-processed into a shared archive file that can be memory-mapped at runtime.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <env>
        <BP_JVM_CDS_ENABLED>true</BP_JVM_CDS_ENABLED>
      </env>
    </image>
  </configuration>
</plugin>
```

App CDS (2)

To create the archive, two additional JVM flags must be specified:

- `-XX:ArchiveClassesAtExit=application.jsa` : creates the CDS archive on exit
- `-Dspring.context.exit=onRefresh` : starts and then immediately exits

Once the archive is available, the JVM can be started with the additional flag:

- `-XX:SharedArchiveFile=application.jsa` : to use it

```
sdk use java 17.0.18-librca
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	80s	450MB	2.64s	236MB	257/s	344MB	179ms	291ms	402ms	703ms	993ms
Java 21	79s	477MB	2.635s	246MB	280/s	364MB	121ms	280ms	392ms	683ms	915ms

I ❤️ Spring Boot  & Buildpacks 

AOT Caching (Java 25+ Project Leyden)

AOT Caching

Project Leyden is an OpenJDK project that aims to improve the startup time and reduce the memory footprint of Java applications by introducing AOT (Ahead-of-Time) compilation and class data sharing (CDS) enhancements.

Training the AOT cache requires two additional JVM flags:

```
-XX:AOTCacheOutput=app.aot -Dspring.context.exit=onRefresh
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 25	83s	570MB	1.98s	240MB	301/s	427MB	117ms	218ms	379ms	597ms	807ms

Other Runtimes

Eclipse OpenJ9

Unleash the power of Java

Optimized to run Java™ applications cost-effectively in the cloud, Eclipse OpenJ9™ is a fast and efficient JVM that delivers power and performance when you need it most.

Optimized for the Cloud, for microservices and monoliths too!

Faster Startup

Faster Ramp-up, when deployed to cloud

Smaller

Our Story

Eclipse OpenJ9

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
        <buildpacks>
            <buildpack>paketobuildpacks/eclipse-openj9:latest</buildpack>
            <!-- Used to inherit all the other Java buildpacks -->
            <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
        </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	73s	266MB	5.114s	180MB	97/s	277MB	499ms	621ms	864ms	1233ms	1556ms
Java 21	71s	284MB	5.373s	185MB	97/s	278MB	500ms	663ms	860ms	1183ms	1506ms
Java 25	71s	283MB	5.756s	185MB	89/s	271MB	575ms	694ms	864ms	1159ms	1366ms

GraalVM

GraalVM CE

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
        <buildpacks>
            <buildpack>paketobuildpacks/graalvm:latest</buildpack>
            <!-- Used to inherit all the other Java buildpacks -->
            <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
        </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	83s	678MB	5.136s	292MB	185/s	385MB	213ms	394ms	582ms	914ms	1272ms
Java 21	81s	663MB	5.118s	311MB	190/s	408MB	204ms	391ms	539ms	934ms	1268ms
Java 25	82s	851MB	5.167s	322MB	210/s	420MB	200ms	318ms	499ms	805ms	1112ms

Oracle JDK

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
        <buildpacks>
            <buildpack>paketobuildpacks/oracle:latest</buildpack>
            <!-- Used to inherit all the other Java buildpacks -->
            <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
        </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21	70s	448MB	4.908s	274MB	240/s	382MB	190ms	299ms	431ms	746ms	1015ms
Java 25	70s	492MB	4.96s	273MB	259/s	446MB	183ms	290ms	400ms	685ms	893ms

Azul Zulu

Azul Zulu

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
        <buildpacks>
            <buildpack>paketobuildpacks/azul-zulu:latest</buildpack>
            <!-- Used to inherit all the other Java buildpacks -->
            <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
        </buildpacks>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	66s	243MB	5.115s	257MB	233/s	368MB	192ms	301ms	442ms	788ms	1039ms
Java 21	66s	267MB	4.855s	278MB	259/s	379MB	183ms	291ms	402ms	701ms	988ms
Java 25	65s	301MB	4.998s	272MB	276/s	385MB	158ms	284ms	394ms	642ms	899ms

Other Buildpack Builders

Bellsoft Buildpack Builder

Bellsoft provides an optimised builder for Spring Boot applications. It uses the Bellsoft Alpaquita, Liberica JDK and the musl C library. A glibc version is also available.

```
<image>
  <builder>bellsoft/buildpacks.builder:musl</builder>
  <!--builder>bellsoft/buildpacks.builder:glibc</builder-->
</image>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17 (m)	65s	183MB	4.951s	288MB	258/s	387MB	184ms	292ms	405ms	707ms	995ms
Java 21 (m)	63s	191MB	4.69s	298MB	273/s	400MB	171ms	284ms	397ms	692ms	917ms
Java 25 (m)	61s	197MB	4.85s	295MB	293/s	396MB	119ms	241ms	383ms	602ms	807ms
Java 17 (g)	66s	189MB	4.824s	271MB	252/s	375MB	188ms	294ms	406ms	701ms	978ms
Java 21 (g)	66s	197MB	4.698s	296MB	278/s	384MB	153ms	282ms	395ms	687ms	932ms
Java 25 (g)	63s	203MB	4.745s	280MB	282/s	386MB	140ms	279ms	390ms	609ms	807ms

Buildpack Jammy Builder Tiny

It's based on Ubuntu Jammy, and works with current JAVA versions. While the base image (default) is bigger the tiny is intended to be used with GraalVM native images.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <builder>paketobuildpacks/builder-jammy-java-tiny</builder>
    </image>
  </configuration>
</plugin>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	66s	280MB	5.059s	257MB	233/s	371MB	193ms	302ms	477ms	781ms	1048ms
Java 21	65s	307MB	5.323s	275MB	259/s	380MB	181ms	291ms	405ms	701ms	992ms
Java 25	65s	376MB	4.963s	270MB	252/s	448MB	184ms	294ms	408ms	705ms	966ms

Buildpack Custom Distroless Builder

Distroless images are minimal base images that contain only the necessary components to run an application, without any additional packages or tools. This can help reduce the attack surface and improve security.

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <builder>ghcr.io/patbaumgartner/distroless-buildpack-builder:latest</builder>
    </image>
  </configuration>
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	97s	298MB	5.236s	260MB	230/s	357MB	194ms	302ms	441ms	764ms	1017ms
Java 21	79s	355MB	4.983s	275MB	257/s	385MB	181ms	292ms	406ms	705ms	994ms
Java 25	75s	424MB	4.981s	270MB	264/s	446MB	181ms	289ms	400ms	683ms	869ms

GraalVM Native Image

GraalVM Native Image (CE & Oracle)

A native image is a technology for building Java code into a standalone executable. This executable contains the application classes, classes from its dependencies, runtime library classes and statically linked native code from the JDK. The JVM is packaged in the native image, so there's no need for a Java Runtime Environment on the target system, but the build artifact is platform dependent.

```
mvn -Pnative spring-boot:build-image

docker run -p 8080:8080 -t spring-petclinic-native:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
CE 25	245s	233MB	0.251s	204MB	1055/s	321MB	45ms	70ms	86ms	113ms	146ms
OG 25	340s	231MB	0.183s	178MB	1260/s	319MB	32ms	60ms	77ms	104ms	140ms

CRaC - OpenJDK

CRaC - OpenJDK (1)

CRaC (Coordinated Restore at Checkpoint) is a feature that allows you to take a snapshot of the state of a Java application and restart it from that state.

Currently only available from:

- Azul Zulu
- Bellsoft Liberica

The application starts within milliseconds!

.. with many other challenges like: embedded configs and secrets, same OS (Linux) and CPU Architecture, same JVM versions, and more ...

CRaC - OpenJDK (2)

```
export JAVA_HOME=/opt/openjdk-17-crac+6_linux-x64/
export PATH=$JAVA_HOME/bin:$PATH
```

```
mvn clean verify
```

```
java -XX:CRaCCheckpointTo=crac-files -jar target/spring-petclinic-crac-4.0.0-SNAPSHOT.jar
```

```
jcmd target/spring-petclinic-crac-4.0.0-SNAPSHOT.jar JDK.checkpoint
```

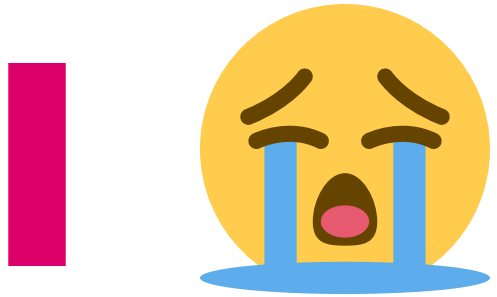
```
java -XX:CRaCRestoreFrom=crac-files
```

CRaC - OpenJDK (3)

CRaC is currently in an experimental state and has the following limitations.

- Since Spring Boot 3.2 CraC support finalised
  - Spring Framework 6.1.0
- Only available for Linux x86 / ARM 64 Bit
- Azul Zulu Build of OpenJDK with CRaC support for development purposes
  - Available for Windows and macOS
  - Simulated checkpoint/restore mechanism for development and testing

Other JVM vendors have similar features, e.g. OpenJ9 with CRIU support.



No CRaC Buildpacks for Java & Spring Boot

Virtual Threads (Java 21+)

Virtual Threads

A thread is the smallest unit of processing that can be scheduled. It runs concurrently with - and largely independently of - other such units. It is an instance of `java.lang.Thread`. There are two types of threads, platform threads and virtual threads.

```
docker run -e spring.threads.virtual.enabled=true \
  -p 8080:8080 -t spring-petclinic-virtual-threads:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 21 🧵	-	-	4.981s	272MB	302/s	370MB	185ms	195ms	201ms	289ms	1496ms
Java 24 🧵	-	-	5.082s	269MB	1721/s	363MB	27ms	30ms	33ms	63ms	355ms
Java 25 🧵	-	-	4.963s	266MB	1713/s	379MB	26ms	29ms	33ms	64ms	940ms
Java 26 🧵	-	-	4.872s	269MB	1728/s	356MB	27ms	31ms	34ms	43ms	64ms

Virtual Threads Java 21

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		4.263s	289MB	1906/s	401MB	21ms	39ms	55ms	81ms	102ms	137ms
4 vCPU		4.631s	290MB	2040/s	400MB	19ms	24ms	42ms	47ms	253ms	615ms
2 vCPU		5.378s	273MB	255/s	377MB	186ms	293ms	405ms	708ms	999ms	1194ms
2 vCPU		4.847s	272MB	312/s	369MB	166ms	193ms	199ms	289ms	1312ms	2003ms
1 vCPU		14.13s	273MB	80/s	367MB	502ms	893ms	1292ms	2198ms	2901ms	3629ms
1 vCPU		14.103s	274MB	110/s	363MB	461ms	498ms	533ms	701ms	2400ms	5527ms
0.5 vCPU		40.16s	271MB	25/s	349MB	1737ms	2735ms	3833ms	6069ms	7916ms	8529ms
0.5 vCPU		40.261s	269MB	37/s	350MB	1396ms	1501ms	1668ms	2402ms	6833ms	7900ms
0.25 vCPU		85.001s	271MB	11/s	340MB	4168ms	6611ms	9303ms	15777ms	19326ms	19326ms
0.25 vCPU		85.136s	271MB	12/s	339MB	4065ms	4462ms	5136ms	8408ms	11641ms	11641ms

Virtual Threads Java 24

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		3.94s	291MB	2050/s	413MB	21ms	34ms	47ms	67ms	83ms	106ms
4 vCPU		3.936s	288MB	2881/s	685MB	11ms	28ms	41ms	76ms	110ms	147ms
2 vCPU		5.08s	272MB	288/s	386MB	138ms	255ms	382ms	604ms	858ms	1048ms
2 vCPU		5.106s	271MB	1722/s	367MB	26ms	30ms	33ms	69ms	428ms	733ms
1 vCPU		15.194s	271MB	83/s	364MB	501ms	837ms	1209ms	2099ms	2934ms	3795ms
1 vCPU		15.053s	269MB	727/s	358MB	58ms	66ms	112ms	181ms	217ms	269ms
0.5 vCPU		43.237s	267MB	25/s	350MB	1734ms	2800ms	3901ms	6118ms	7967ms	9132ms
0.5 vCPU		43.221s	267MB	58/s	351MB	827ms	965ms	1127ms	1638ms	2235ms	2464ms
0.25 vCPU		90.669s	267MB	11/s	327MB	4034ms	6532ms	9566ms	15415ms	20809ms	20809ms
0.25 vCPU		89.97s	265MB	15/s	322MB	3166ms	3634ms	4033ms	7540ms	10232ms	10232ms

Virtual Threads Java 25

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		3.887s	287MB	1924/s	683MB	21ms	38ms	53ms	73ms	88ms	102ms
4 vCPU		3.863s	287MB	2774/s	681MB	5ms	29ms	46ms	85ms	122ms	156ms
2 vCPU		4.901s	270MB	250/s	443MB	187ms	295ms	426ms	675ms	932ms	1117ms
2 vCPU		4.863s	269MB	1757/s	377MB	26ms	28ms	32ms	60ms	937ms	1973ms
1 vCPU		14.072s	270MB	84/s	381MB	500ms	809ms	1233ms	2039ms	2803ms	3728ms
1 vCPU		13.468s	270MB	737/s	355MB	57ms	66ms	111ms	178ms	204ms	263ms
0.5 vCPU		40.636s	266MB	26/s	344MB	1652ms	2667ms	3836ms	6399ms	8938ms	9267ms
0.5 vCPU		41.775s	265MB	60/s	356MB	797ms	903ms	1066ms	1563ms	1667ms	1695ms
0.25 vCPU		86.27s	267MB	11/s	325MB	4034ms	6212ms	8803ms	14479ms	17807ms	17807ms
0.25 vCPU		86.703s	264MB	16/s	324MB	2935ms	3303ms	3634ms	7431ms	14234ms	14234ms

Virtual Threads Java 26

CPU	VT	Startup	Initial RAM	Request/s	RAM	50%	75%	90%	99%	99.9%	99.99%
4 vCPU		3.845s	286MB	1826/s	424MB	23ms	40ms	55ms	75ms	91ms	109ms
4 vCPU		3.797s	285MB	2658/s	438MB	18ms	26ms	40ms	71ms	100ms	127ms
2 vCPU		4.996s	272MB	271/s	388MB	174ms	287ms	398ms	665ms	902ms	1127ms
2 vCPU		4.866s	269MB	1731/s	357MB	27ms	31ms	34ms	44ms	62ms	70ms
1 vCPU		13.319s	271MB	77/s	368MB	560ms	901ms	1307ms	2236ms	2971ms	4040ms
1 vCPU		14.049s	270MB	914/s	352MB	55ms	58ms	62ms	104ms	169ms	186ms
0.5 vCPU		40.558s	267MB	25/s	342MB	1698ms	2677ms	3868ms	6432ms	9100ms	9832ms
0.5 vCPU		41.234s	266MB	64/s	351MB	790ms	897ms	1035ms	1302ms	1499ms	1554ms
0.25 vCPU		86.468s	266MB	11/s	328MB	4200ms	6499ms	9177ms	14391ms	17587ms	17587ms
0.25 vCPU		86.006s	264MB	16/s	319MB	3167ms	3634ms	4129ms	6609ms	7391ms	7391ms

Summary

Summary

- No Optimisations with JDK 17 & JDK 21, ...
- JVM Tuning
- Lazy Spring Beans
- No Spring Boot Actuators
- Fix Spring Boot Config Location
- Disabling JMX
- Dependency Cleanup
- Ahead-of-Time Processing (AOT)
- JLink
- Other JVMs (Eclipse OpenJ9, GraalVM, OpenJDK with CRaC)
- GraalVM Native Image

Conclusions

Conclusions (1)

CPUs

- Your application may not need a full CPU at runtime.
- It will need several CPUs to start as fast as possible (at least 2, 4 is better).
- If you don't mind a slower startup, you can throttle the CPUs below 4.

See: https://spring.io/blog/2018/11/08/spring-boot-in-a-container

Conclusions (2)

Request/s

- Every application is different and has different requirements.
- Proper load testing can help find the optimal configuration for your application.

Conclusions (3)

Other Runtimes

- CRIU Support for OpenJDK and OpenJ9 is promising.
  - Supported by Spring since Spring Boot 3.2 / Spring Framework 6.1
- GraalVM Native Image is a great option for Java applications
  - But build times are long
  - The result is different from what you run in your IDE
- Eclipse OpenJ9 is an excellent option for running applications with less memory
  - But startup times are longer than with HotSpot.
- Depending on the distribution, you may get other exciting features.
  - Oracle GraalVM Enterprise Edition, Azul Platform Prime, IBM Semeru Runtime, ...

Conclusions (4)

Other Ideas

- CRaC (Coordinated Restore at Checkpoint)*
- Tree shaking / Obfuscator such as ProGuard*
- AutoConfiguration Filter -> **spring-boot-autoconfiguration-optimizer**
- More JVM tuning (GC, Memory, etc.)
- Project Leyden

A Few Simple Optimisations Applied

A Few Simple Optimisations Applied

- Dependency Cleanup
  - DB Drivers, Spring Boot Actuator, Jackson, Tomcat Websocket, ...
- Bellsoft Builder (musl) / Base Builder Tiny
- JLink
- JVM Parameters (java-memory-calculator)
- Application Class Data Sharing (AppCDS)
- Spring AOT
- Lazy Spring Beans
- Fixing Spring Boot Config Location
- Virtual Threads

Bellsoft Builder (musl)

```
sdk use java 21.0.10-librca

mvn spring-boot:build-image

docker run -p 8080:8080 \
  -e spring.threads.virtual.enabled=true \
  -e spring.main.lazy-initialization=true \
  -e spring.data.jpa.repositories.bootstrap-mode=lazy \
  -e spring.config.location=classpath:application.properties \
  -t spring-petclinic-optimized-builder-bellsoft-musl:4.0.0-SNAPSHOT
```

	Build	Image	Startup	Initial RAM	Requests/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17	66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17	48s	145MB	3.348s	224MB	263/s	368MB	126ms	290ms	401ms	704ms	995ms
Java 21 🧵	51s	211MB	2.918s	228MB	361/s	370MB	101ms	200ms	301ms	540ms	785ms
Java 25 🧵	57s	318MB	3.003s	228MB	433/s	670MB	97ms	186ms	278ms	463ms	610ms

Base Builder Tiny

```
docker run -p 8080:8080 \
  -e spring.threads.virtual.enabled=true \
  -e spring.main.lazy-initialization=true \
  -e spring.data.jpa.repositories.bootstrap-mode=lazy \
  -e spring.config.location=classpath:application.properties \
  -t spring-petclinic-optimized-builder-tiny-virtual-threads:4.0.0-SNAPSHOT
```

	VT	AC	COH	Build	Image	Startup	In. RAM	R/s	RAM	50%	75%	90%	99%	99.9%
⏱️ Java 17				66s	279MB	5.335s	259MB	235/s	360MB	193ms	301ms	453ms	761ms	1004ms
Java 17				60s	311MB	3.14s	216MB	324/s	346MB	105ms	209ms	333ms	602ms	885ms
Java 21	🧵			58s	318MB	2.921s	222MB	346/s	360MB	103ms	204ms	308ms	589ms	798ms
Java 25	🧵			61s	344MB	3.079s	221MB	438/s	645MB	96ms	185ms	279ms	483ms	657ms
Java 25		♻️		62s	355MB	3.829s	252MB	349/s	525MB	107ms	203ms	304ms	513ms	702ms
Java 25	🧵	♻️		62s	355MB	3.383s	233MB	2256/s	350MB	21ms	22ms	25ms	36ms	651ms
Java 25		♻️	📦	58s	547MB	3.927s	241MB	367/s	519MB	105ms	199ms	297ms	501ms	694ms
Java 25	🧵	♻️	📦	58s	547MB	3.394s	227MB	2256/s	337MB	20ms	22ms	25ms	36ms	585ms

Did I miss something? 🧐

Let me/us know! 🙋

… or get in touch later! 🙊

Feedback is a Gift

Your turn 👇

👉 **Use the Devoxx Compagnion App**

- 🐛 What didn’t work (aka bugs)?
- 🤯 What was confusing (missing docs)?
- 🚀 What should I refactor next?

No feedback = it works on your machine™

Spring Boot in the Cloud

Advanced Optimization Deep Dive

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com
bit.ly/48i5U9x